



Logic

SE-350/SE-450, Spring-2026
Artificial Intelligence (AI)

Dr. Kanwal Yousaf

Department of Software Engineering
UET Taxila

e-mail: kanwal.yousaf@uettaxila.edu.pk

Some slides material from
Chen-Yu Wei, University of Virginia

Knowledge-Based Agents

function KB-AGENT(*percept*) **returns** an *action*
 persistent: *KB*, a knowledge base
 t, a counter, initially 0, indicating time

 TELL(*KB*, MAKE-PERCEPT-SENTENCE(*percept*, *t*))
 action ← ASK(*KB*, MAKE-ACTION-QUERY(*t*))
 TELL(*KB*, MAKE-ACTION-SENTENCE(*action*, *t*))
 t ← *t* + 1
 return *action*

Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

Wumpus World

- Performance measure**

- Gold +1000, death -1000, -1 per step, -10 for using the arrow

- Environment**

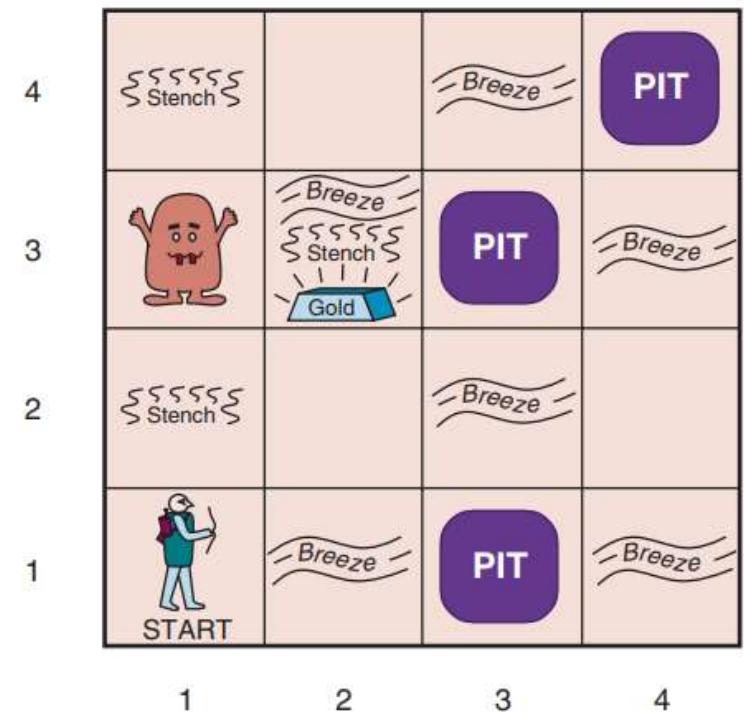
- Perceive stench if adjacent to wumpus
- Perceive breeze if adjacent to pit
- Perceive glitter if in the square of gold
- Can grab gold if in the square of gold
- Can shoot and kill wumpus if you're facing it (shooting uses up the only arrow)
- Die if entering a square with pit or living wumpus

- Actions**

- Left turn, right turn, forward, grab, shoot

- Sensors**

- Breeze, glitter, stench, bump, scream



Exploring a wumpus world

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 B OK A	2,2	3,2	4,2
1,1 ↑ A OK	2,1 OK	3,1	4,1

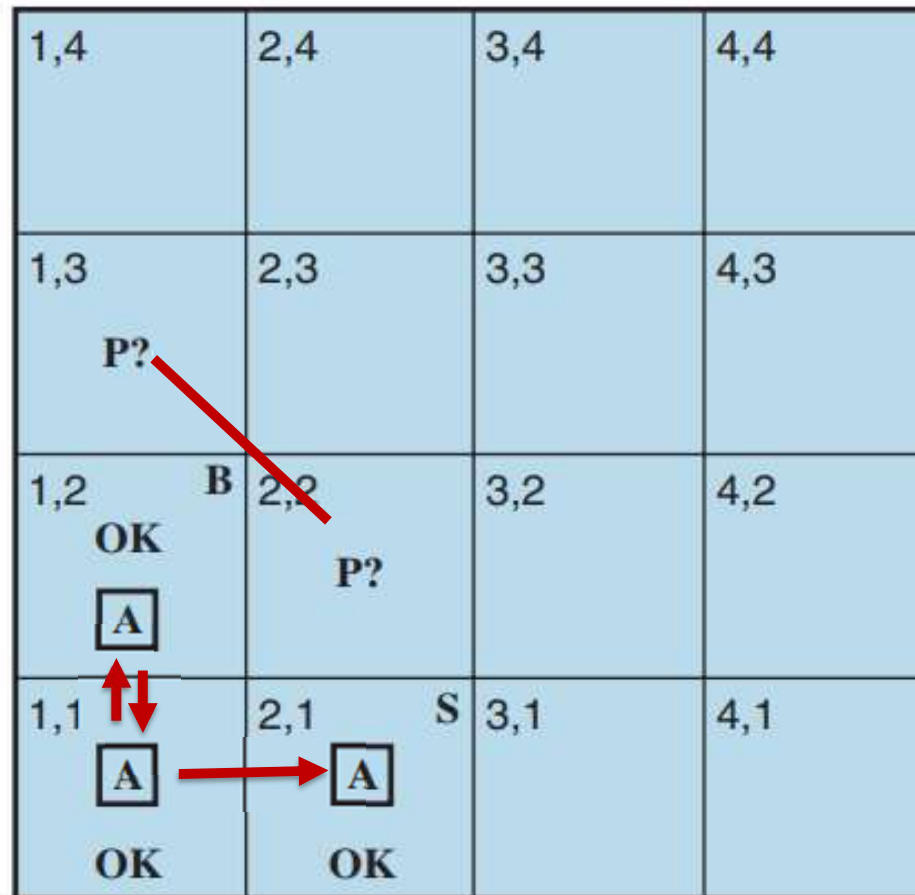
- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)

1,4	2,4	3,4	4,4
1,3 P?	2,3	3,3	4,3
1,2 OK A	B 2,2 P?	3,2	4,2
1,1 A OK	2,1 OK	3,1	4,1

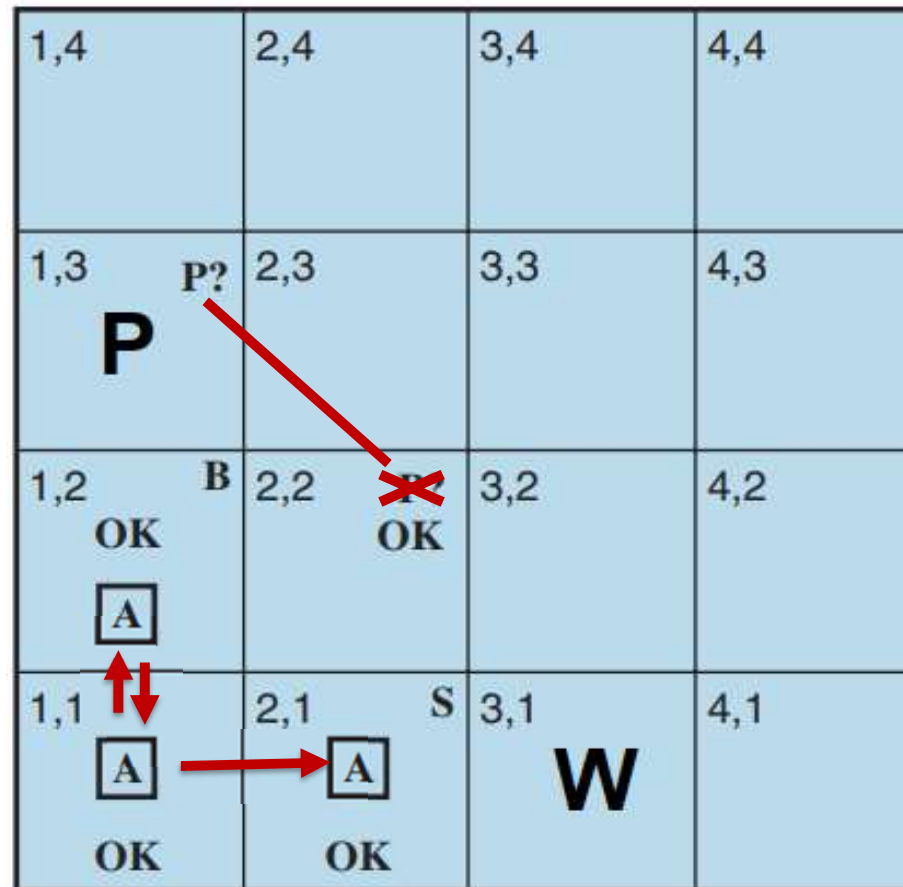
- A = Agent
- B = Breeze
- G = Glitter, Gold
- OK = Safe square
- P = Pit
- S = Stench
- V = Visited
- W = Wumpus

Exploring a wumpus world (cont...)



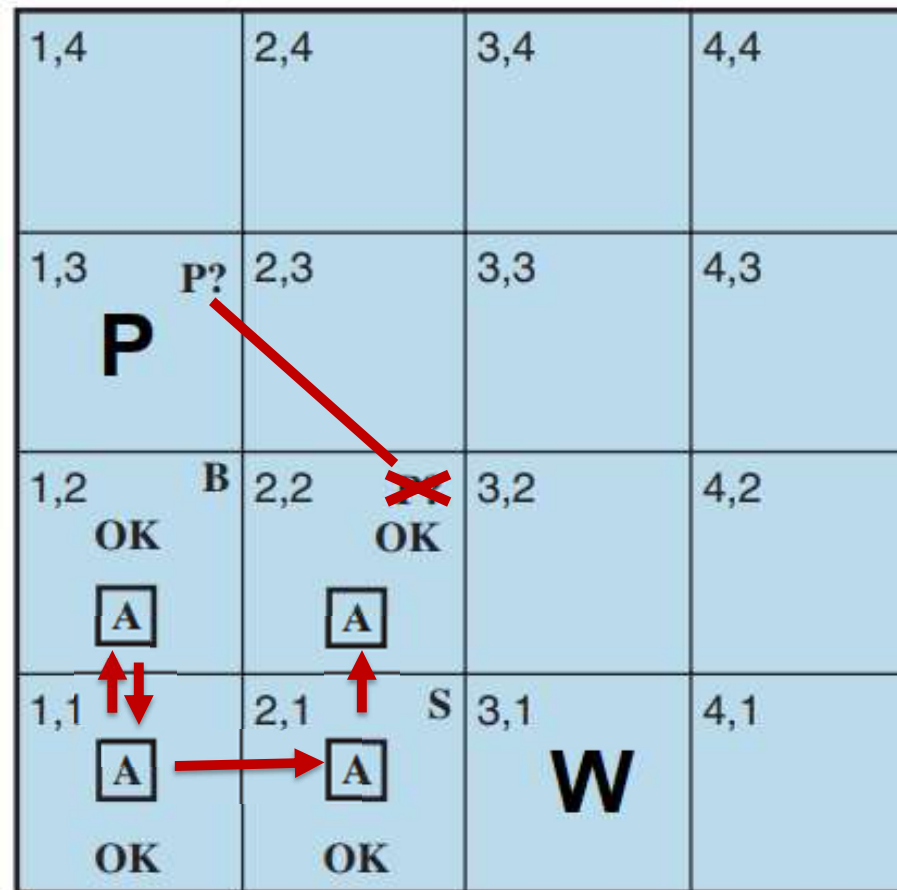
- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)



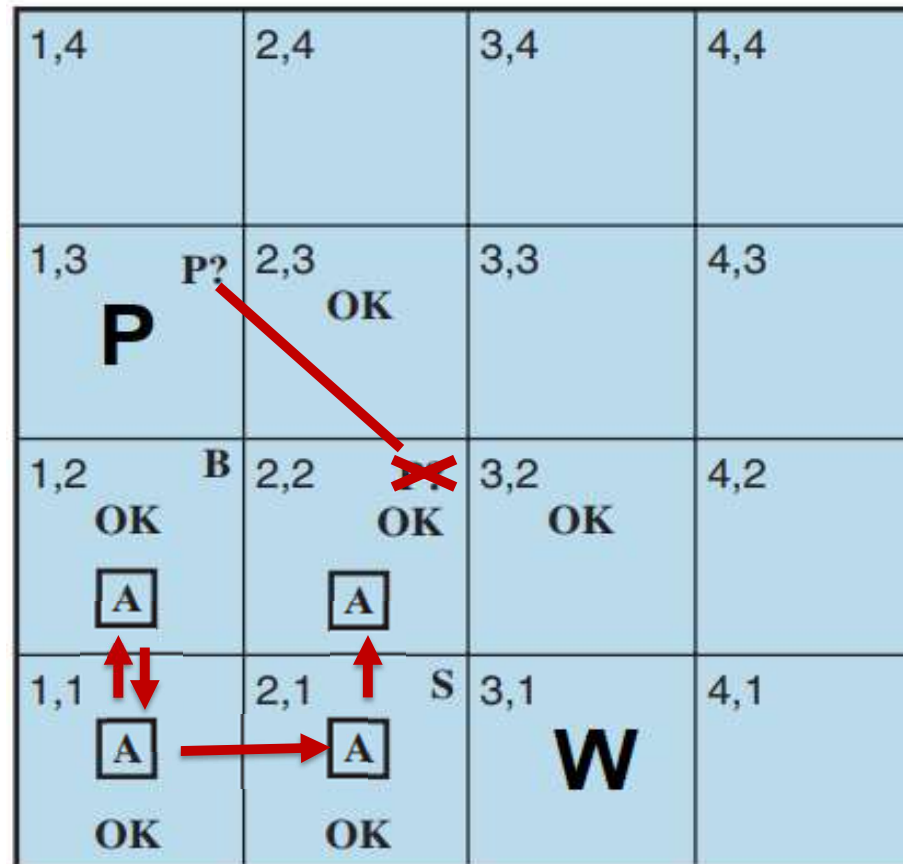
- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)



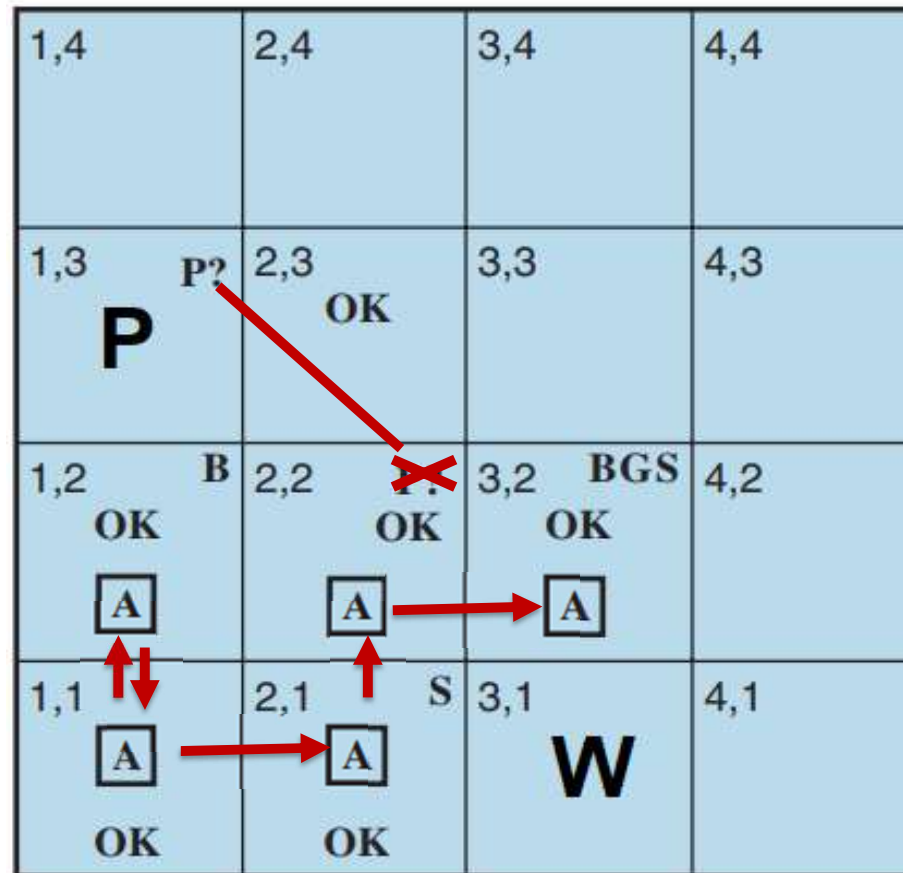
- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)



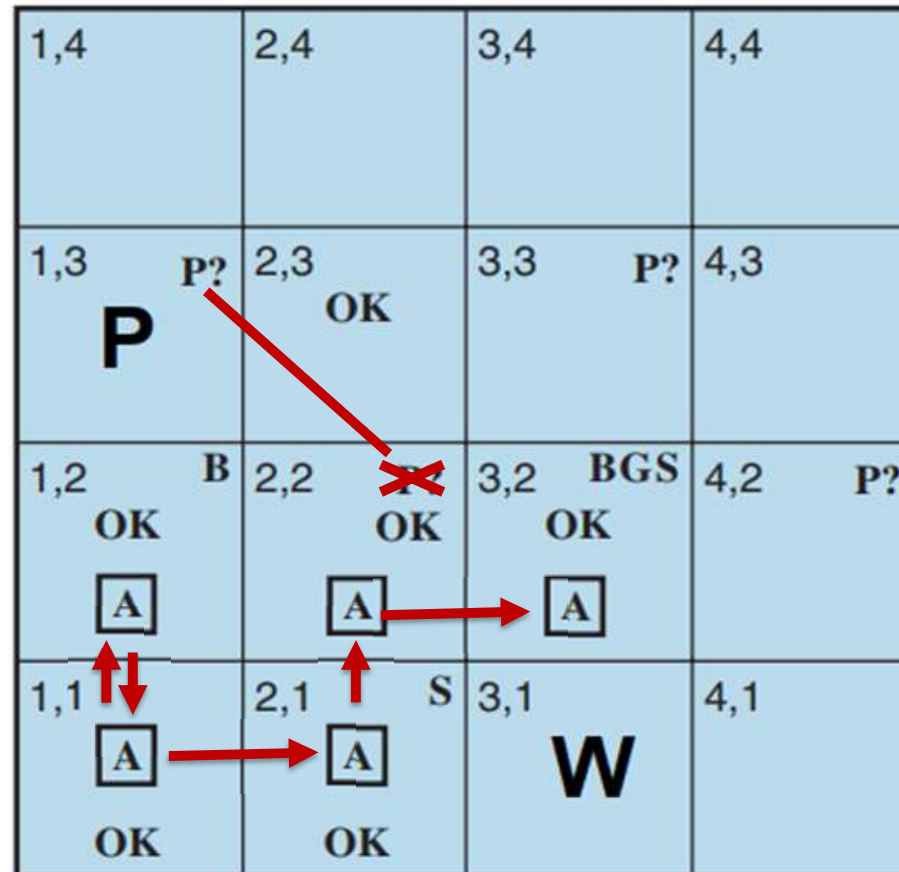
- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)



- A** = Agent
- B** = Breeze
- G** = Glitter, Gold
- OK** = Safe square
- P** = Pit
- S** = Stench
- V** = Visited
- W** = Wumpus

Exploring a wumpus world (cont...)



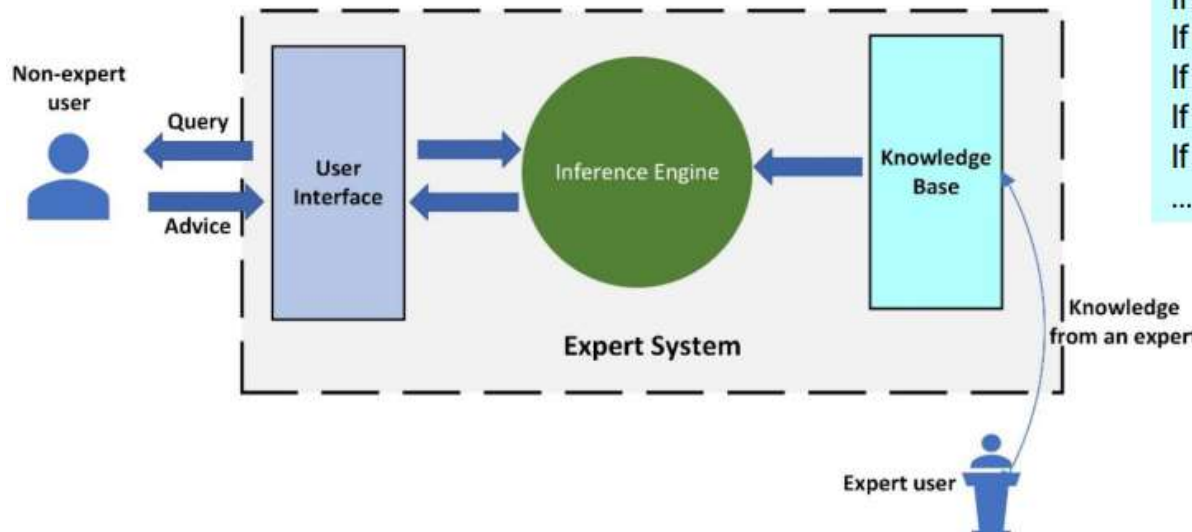
- [A] = Agent
- B = Breeze
- G = Glitter, Gold
- OK = Safe square
- P = Pit
- S = Stench
- V = Visited
- W = Wumpus



Systems with Logical Reasoning

- Knowledge base
 - Consists of some prior knowledge
- Inference engine
 - Derive new knowledge or make some claims
- User Interaction
 - Tell information
 - Ask question

Example: Expert System



Knowledge Base

If has_hair, then mammal.
 If mammal and has_hooves, then ungulate.
 If has_feathers, then bird.
 If mammal and carnivore and has_dark_spots, then cheetah.
 If mammal and carnivore and has_black_stripes, then tiger.
 If bird and does_not_fly and has_long_neck, then ostrich.


```

File Edit Settings Run Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 9.2.6)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license, for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic), or ?- apropos(Word).

?- go.
Does the animal have hair? yes.
Does the animal eat meat? |: no.
Does the animal have pointed teeth? |: no.
Does the animal have hooves? |: yes.
Does the animal have a long neck? |: yes.
Does the animal have long legs? |: yes.
I guess that the animal is: giraffe
true.
?- █
  
```

User Interaction

Example: Wumpus World

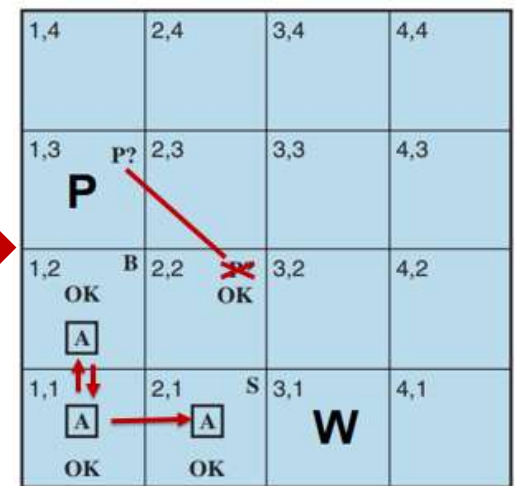
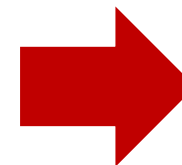
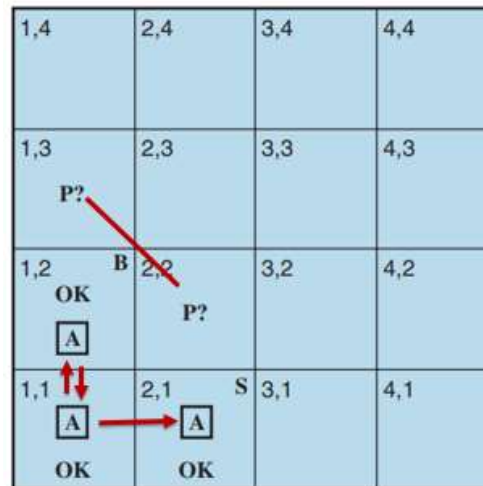
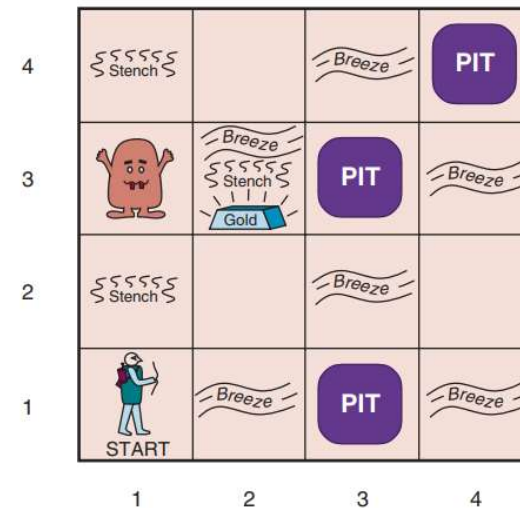
- Knowledge base**

- Perceive stench if adjacent to wumpus
- Perceive breeze if adjacent to pit
- Perceive glitter if in the square of gold
- ...

- User interaction**

- Tell the logic system whether stench, breeze, glitter is perceived
- Ask for the next action

- Inference Engine**





Propositional Logic: A Very Simple Logic



Ingredients of propositional logic: Sentence

Sentence

- Knowledge base consists of “sentences”
- Inference algorithm derives new “sentences” and add them to the knowledge base
- **Example:**
 - $KB = \{ \text{“Rain} \rightarrow \text{Wet”}, \text{“Rain”} \}$
 - Inference algorithm derives a new sentence **“Wet”** based on KB
 - Now KB becomes
 - $KB = \{ \text{“Rain} \rightarrow \text{Wet”}, \text{“Rain”}, \text{“Wet”} \}$



Ingredients of propositional logic: Syntax

- Define what are valid sentences
- E.g., syntax in **python**:
 - “for x in range(10): ” **Valid**
 - “for x range(10): ” **Invalid** (the python interpreter cannot understand)
- E.g. syntax in **math**:
 - “ $x + y = 5$ ” **Valid**
 - “ $x\ 5 = y +$ ” **Invalid**



Ingredients of propositional logic: Syntax (cont...)

- A proposition symbols X is a sentence
(a propositional symbol is a Boolean variable)
 - If α is a sentence then $\neg\alpha$ is a sentence
 - If α and β are sentences then $\alpha\wedge\beta$ is a sentence
 - If α and β are sentences then $\alpha\vee\beta$ is a sentence
 - If α and β are sentences then $\alpha\Rightarrow\beta$ is a sentence
 - If α and β are sentences then $\alpha\Leftrightarrow\beta$ is a sentence
- The $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ symbols have no meaning here. Their meanings are specified by the “semantics” of logic

OPERATOR PRECEDENCE

$\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Ingredients of propositional logic: Semantics

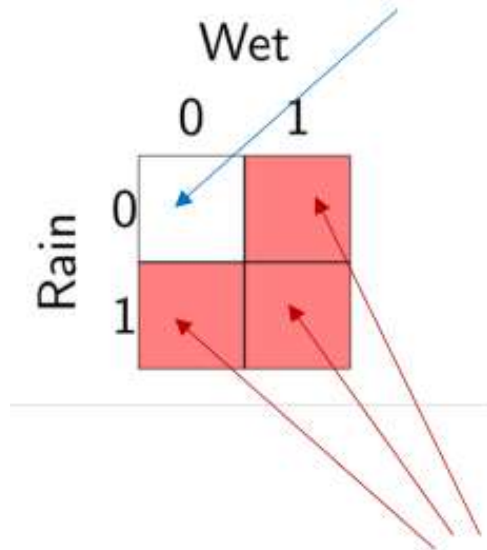
- Let's first define "models". A model is a configuration of the world.
- In propositional logic, a model is an **assignment of truth values** to propositional symbols.
- E.g., There are four possible models in the raining example:

		Wet	
		0	1
Rain	0		
	1		

Ingredients of propositional logic: Semantics (cont...)

- $f = \text{Rain} \vee \text{Wet}$

models where the sentence f is false



P	Q	(P $\vee$ Q)
T	T	T
T	F	T
F	T	T
F	F	F

models where the sentence f is true

Ingredients of propositional logic: Semantics (cont...)

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>true</i>
<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>false</i>
<i>true</i>	<i>false</i>	<i>false</i>	<i>false</i>	<i>true</i>	<i>false</i>	<i>false</i>
<i>true</i>	<i>true</i>	<i>false</i>	<i>true</i>	<i>true</i>	<i>true</i>	<i>true</i>

Ingredients of propositional logic: Semantics (cont...)

- $f: (Rain \vee Wet) \Rightarrow Unhappy$

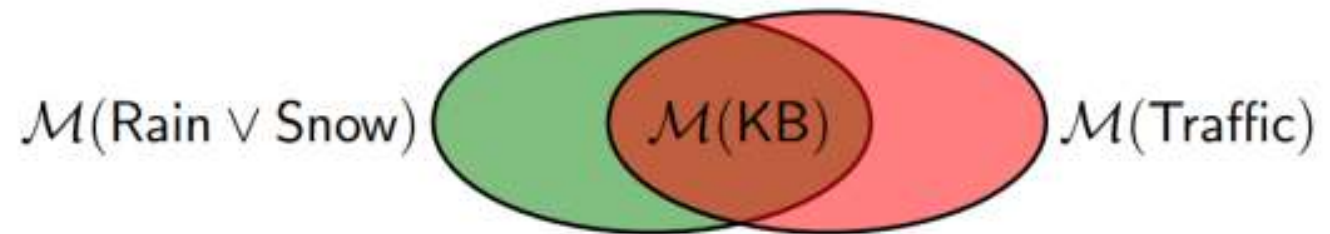
		Unhappy	
		0	1
Rain, Wet	00		
	01		
	10		
	11		

$\mathcal{M}(f)$: the set of models where sentence f is true.

Ingredients of propositional logic: Knowledge Base

- Knowledge base = a collection of sentences

Let $KB = \{Rain \vee Snow, Traffic\}$.



Ingredients of propositional logic: Knowledge Base (cont...)

$\mathcal{M}(\text{Rain})$

		Wet	
		0	1
Rain	0		
	1		

$\mathcal{M}(\text{Rain} \rightarrow \text{Wet})$

		Wet	
		0	1
Rain	0		
	1		

Adding more formulas to the knowledge base:

$$KB \rightarrow KB \cup \{f\}$$

$\mathcal{M}(\{\text{Rain}, \text{Rain} \rightarrow \text{Wet}\})$

		Wet	
		0	1
Rain	0		
	1		

Shrinks the set of models:

$$KB \rightarrow \mathcal{M}(KB) \cap \mathcal{M}\{f\}$$

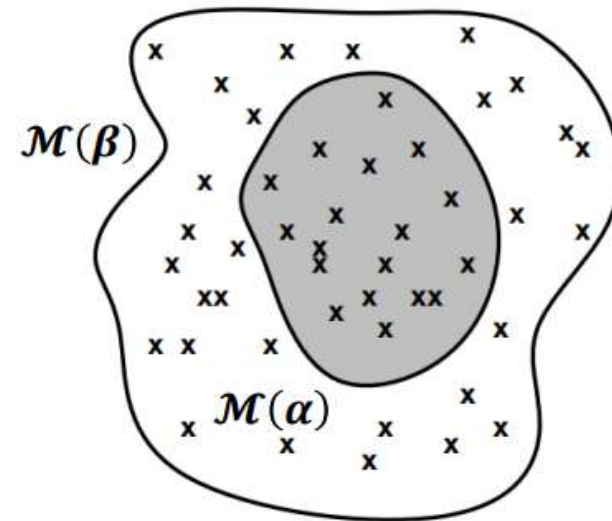


Recap: Propositional Logic

- Sentence: propositional symbols, or their negations ($\neg$), or their combinations through $\wedge, \vee, \Rightarrow, \Leftrightarrow$.
- Models: An assignment of truth values to propositional symbols.
- Knowledge base: a set of sentences
- $\mathcal{M}(f)$: the set of models where sentence f is true.

Entailment

- Sentence α entails sentence β means that (in high level) sentence β follows logically from sentence α
- Denoted as $\alpha \models \beta$
- $\alpha \models \beta$ if and only if $\mathcal{M}(\alpha) \subseteq \mathcal{M}(\beta)$
- **Example:** $\text{Rain} \wedge \text{Snow} \models \text{Snow}$





Inference Algorithms

- Given KB and α , the algorithm tries to derive sentence α .
- If an algorithm $\mathcal{A}$ is able to derive α from KB, we write $KB \vdash_{\mathcal{A}} \alpha$
 - This is different from $KB \models \alpha$,
- Soundness (correctness)
 - The algorithm can only derive α when α is entailed by KB.
 - In other words: If $KB \vdash_{\mathcal{A}} \alpha$, then $KB \models \alpha$
- Completeness
 - For any α that KB entails, the algorithm is able to derive α .
 - In other words: If $KB \models \alpha$, then if $KB \vdash_{\mathcal{A}} \alpha$



A (Simple) Inference Algorithm: Model Checking

function TT-ENTAILS?(KB, α) **returns** *true* or *false*

inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$symbols \leftarrow$ a list of the proposition symbols in KB and α

return TT-CHECK-ALL($KB, \alpha, symbols, \{\}$)

function TT-CHECK-ALL($KB, \alpha, symbols, model$) **returns** *true* or *false*

if EMPTY?($symbols$) **then**

if PL-TRUE?($KB, model$) **then return** PL-TRUE?($\alpha, model$)

else return *true* // when KB is false, always return true

else

$P \leftarrow$ FIRST($symbols$)

$rest \leftarrow$ REST($symbols$)

return (TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = true\}$)

and

 TT-CHECK-ALL($KB, \alpha, rest, model \cup \{P = false\}$))

A (Simple) Inference Algorithm: Model Checking (cont...)

Model Checking (KB, α):

Let $\mathcal{M}$ be the set of all possible models

($|\mathcal{M}| = 2^N$ if there are N propositional symbols in $\text{KB} \cup \{\alpha\}$)

For $m \in \mathcal{M}$:

If **KB is True in m** and **α is False in m** : **return False**

return True



Theorem Proving

- **Idea:** Instead of checking all models, will just perform manipulations **on the sentence level**.



Inference Rules

- Modus Ponens (Latin for *mode that affirms*)

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta}$$

$$\text{or, } \frac{\alpha_1, \alpha_2, \dots, \alpha_k, (\neg \alpha_1 \vee \neg \alpha_2 \vee \dots \vee \neg \alpha_k) \Rightarrow \beta}{\beta}$$

- And Eliminations

$$\frac{\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k}{\alpha_i}$$

Standard Logical Equivalence

(can be applied in any steps in the inference algorithm)

$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	implication elimination
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	De Morgan
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	De Morgan
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	distributivity of $\vee$ over $\wedge$

Figure 7.11 Standard logical equivalences. The symbols α , β , and γ stand for arbitrary sentences of propositional logic.



Inference Rules

Example: KB = {Rain $\Rightarrow$ Wet, Wet $\Rightarrow$ Unhappy, Rain}, α = Unhappy

- Applying Modus Ponens on KB
(i.e., try to match sentences in KB with premises α and β)

$$\frac{\text{Rain}, \text{Rain} \Rightarrow \text{Wet}}{\text{Wet}}$$

- KB = {Rain $\Rightarrow$ Wet, Wet $\Rightarrow$ Unhappy, Rain, Wet}
- Applying Modus Ponens on KB

$$\frac{\text{Wet}, \text{Wet} \Rightarrow \text{Unhappy}}{\text{Unhappy}}$$

Modus Ponens:

$$\frac{\alpha_1 \dots, \alpha_k, (\alpha_1 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta}$$

Forward Inference

Input: KB, α , $\mathfrak{I}$ = a set of inference rule

If $\alpha \in \text{KB}$: **return** True

Repeat:

Choose a set of sentences $\alpha_1, \dots, \alpha_k \in \text{KB}$ such that

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k}{\beta}$$

matches a rule in $\mathfrak{I}$, and $\beta \notin \text{KB}$.

If $\beta = \alpha$: **return** True

If such $(\alpha_1, \alpha_2, \dots, \alpha_k, \beta)$ does not exist: **return** False

Add β to KB.



Forward Inference (cont...)

- Forward inference is a search problem
 - What are the states, actions, successor function, and goal test?
 - Algorithms introduced for search problems can be applied here.
- Is the forward inference algorithm sound?
 - Yes, as long as all inference rules you use are sound
- Is forward inference complete?

Forward Inference (cont...)

Example:

- $KB = \{\text{Rain} \Rightarrow \text{Wet}, \text{Rain} \vee \text{Shine}, \text{Wet} \vee \text{Shine} \Rightarrow \text{Happy}\}$
 - $\alpha = \text{Happy}$
 - Use Forward Inference algorithm with $\mathcal{I} = \{\text{Modus Ponens}\}$
-
- Can KB entail α ?
 - Can the algorithm derive α from KB?
 - Forward Inference with Modus Ponens is **sound** but **not complete**



A Sound and Complete Algorithm?

- **Fact 1.** If KB only consists of **Horn clauses**,
 - then Forward Inference with **Modus Ponens** is sound and complete.
- **Fact 2.** In general, Forward Inference with **Resolution** is sound and complete.



Horn Clauses + Modus Ponens is Complete

- **Horn clause:** sentence that have the following forms

$$X_1 \wedge X_2 \wedge \cdots \wedge X_{k-1} \Rightarrow X_k$$

$$X_1 \wedge X_2 \wedge \cdots \wedge X_k \Rightarrow \textit{False}$$

$$\neg X_1 \vee \neg X_2 \vee \cdots \vee \neg X_{k-1} \vee X_k$$

$$\neg X_1 \vee \neg X_2 \vee \cdots \vee \neg X_k$$

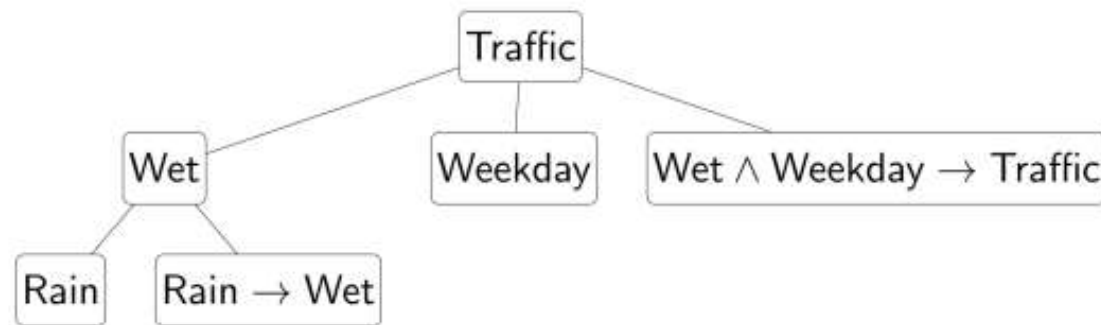
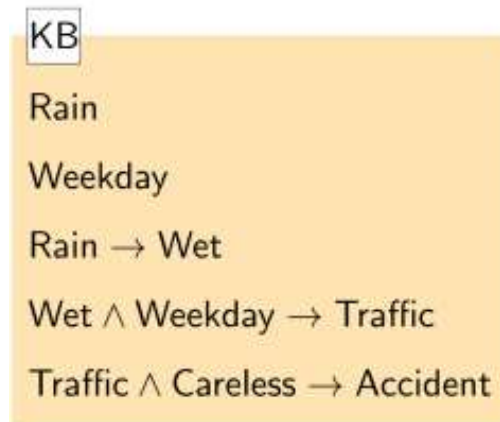
Disjunction with only one positive symbol

(Definite clause)

Disjunction with no positive symbol

(Goal clause)

Horn Clauses + Modus Ponens is Complete (cont...)



- **Intuition:** The inference procedure of horn clauses is direct, in the sense that there is no branching.
- Horn clause: $\text{Rain} \wedge \text{Snow} \rightarrow \text{Dark} \wedge \text{Traffic}$
- Non- horn clause: $\text{Wet} \rightarrow \text{Rain} \vee \text{Snow}$

Has to branch into the cases $\neg \text{Rain}$, $\neg \text{Snow}$ etc.



```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
           q, the query, a proposition symbol
  count  $\leftarrow$  a table, where count[c] is initially the number of symbols in clause c's premise
  inferred  $\leftarrow$  a table, where inferred[s] is initially false for all symbols
  queue  $\leftarrow$  a queue of symbols, initially symbols known to be true in KB

  while queue is not empty do
    p  $\leftarrow$  POP(queue)
    if p = q then return true
    if inferred[p] = false then
      inferred[p]  $\leftarrow$  true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to queue
  return false
```

Figure 7.15 The forward-chaining algorithm for propositional logic. The *queue* keeps track of symbols known to be true but not yet “processed.” The *count* table keeps track of how many premises of each implication are not yet proven. Whenever a new symbol *p* from the agenda is processed, the count is reduced by one for each implication in whose premise *p* appears (easily identified in constant time with appropriate indexing.) If a count reaches zero, all the premises of the implication are known, so its conclusion can be added to the agenda. Finally, we need to keep track of which symbols have been processed; a symbol that is already in the set of inferred symbols need not be added to the agenda again. This avoids redundant work and prevents loops caused by implications such as $P \Rightarrow Q$ and $Q \Rightarrow P$.

A pseudocode for Forward Inference with Modus Ponens (this algorithm is also called **Forward Chaining**). This pseudocode assumes that all sentences are definite clauses (but it's easy to extend it to handle goal clauses as well).

The time complexity is linear in the “**size of KB**”, i.e., the sum of the lengths of all sentences in KB.

General Case: Resolution is Complete

- Resolution**

$$\frac{\alpha_1 \vee \alpha_2 \vee \cdots \vee \alpha_k \vee p, \quad \neg p \vee \beta_1 \vee \beta_2 \vee \cdots \vee \beta_m}{\alpha_1 \vee \alpha_2 \vee \cdots \vee \alpha_k \vee \beta_1 \vee \beta_2 \vee \cdots \vee \beta_m}$$

- Example**

$$\frac{\text{Rain} \vee \text{Shine}, \quad \neg \text{Rain} \vee \text{Wet}}{\text{Shine} \vee \text{Wet}}$$

Converting Sentences to CNF Before Applying Resolution

Conversion rules:

- Eliminate $\leftrightarrow$: $\frac{f \leftrightarrow g}{(f \rightarrow g) \wedge (g \rightarrow f)}$
- Eliminate $\rightarrow$: $\frac{f \rightarrow g}{\neg f \vee g}$
- Move $\neg$ inwards: $\frac{\neg(f \wedge g)}{\neg f \vee \neg g}$
- Move $\neg$ inwards: $\frac{\neg(f \vee g)}{\neg f \wedge \neg g}$
- Eliminate double negation: $\frac{\neg \neg f}{f}$
- Distribute $\vee$ over $\wedge$: $\frac{f \vee (g \wedge h)}{(f \vee g) \wedge (f \vee h)}$

Conjunctive Normal Form (CNF)



Converting Sentences to CNF: Example

Initial formula:

$$(\text{Summer} \rightarrow \text{Snow}) \rightarrow \text{Bizzare}$$

Remove implication ($\rightarrow$):

$$\neg(\neg\text{Summer} \vee \text{Snow}) \vee \text{Bizzare}$$

Push negation ($\neg$) inwards (de Morgan):

$$(\neg\neg\text{Summer} \wedge \neg\text{Snow}) \vee \text{Bizzare}$$

Remove double negation:

$$(\text{Summer} \wedge \neg\text{Snow}) \vee \text{Bizzare}$$

Distribute $\vee$ over $\wedge$:

$$(\text{Summer} \vee \text{Bizzare}) \wedge (\neg\text{Snow} \vee \text{Bizzare})$$



Resolution-Based Inference Algorithm

- Note that $\mathbf{KB} \models \alpha$ is equivalent to $\mathcal{M}(\mathbf{KB} \wedge \neg \alpha) = \text{empty set}$

$\mathbf{KB}' \leftarrow \mathbf{KB} \cup \{\neg \alpha\}$

Convert all sentences in $\mathbf{KB}'$ to CNF

Repeatedly apply Resolution Rule until

1) False is derived $\rightarrow$ return $\mathbf{KB} \models \alpha$

2) No new sentence can be derived $\rightarrow$ return $\mathbf{KB} \not\models \alpha$

function PL-RESOLUTION(KB, α) **returns** *true* or *false*
inputs: KB , the knowledge base, a sentence in propositional logic
 α , the query, a sentence in propositional logic

$clauses \leftarrow$ the set of clauses in the CNF representation of $KB \wedge \neg \alpha$
 $new \leftarrow \{ \}$
while *true* **do**
 for each pair of clauses C_i, C_j **in** $clauses$ **do**
 $resolvents \leftarrow$ PL-RESOLVE(C_i, C_j)
 if $resolvents$ contains the empty clause **then return** *true*
 $new \leftarrow new \cup resolvents$
 if $new \subseteq clauses$ **then return** *false*
 $clauses \leftarrow clauses \cup new$

Figure 7.13 A simple resolution algorithm for propositional logic. PL-RESOLVE returns the set of all possible clauses obtained by resolving its two inputs.

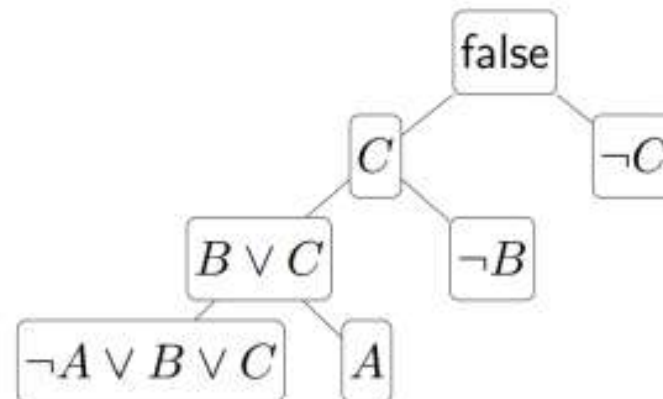
Resolution-Based Inference Algorithm (cont...)

$$KB' = \{A \rightarrow (B \vee C), A, \neg B, \neg C\}$$

Convert to CNF:

$$KB' = \{\neg A \vee B \vee C, A, \neg B, \neg C\}$$

Repeatedly apply **resolution** rule:



Conclusion: ***KB entails f***

Time Complexity

- Modus Ponens

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta}$$

Each rule application adds sentence with **one** propositional symbol

→ **linear time**

- Resolution

$$\frac{\alpha_1 \vee \alpha_2 \vee \dots \vee \alpha_k \vee p, \quad \neg p \vee \beta_1 \vee \beta_2 \vee \dots \vee \beta_m}{\alpha_1 \vee \alpha_2 \vee \dots \vee \alpha_k \vee \beta_1 \vee \beta_2 \vee \dots \vee \beta_m}$$

Each rule application adds sentence with **many** propositional symbol

→ **exponential time**

Recap



	Modus Ponens	Resolution
Sound?	Yes	Yes
Complete?	No	Yes
Complete for horn clauses?	Yes	Yes
Time complexity	linear	exponential



First-Order Logic (FOL)



Limitations of Propositional Logic

Alice and Bob both know logic.

$\text{AliceKnowsLogic} \wedge \text{BobKnowsLogic}$

Every student knows logic.

$\text{AliceIsStudent} \rightarrow \text{AliceKnowsLogic}$

$\text{BobIsStudent} \rightarrow \text{BobKnowsLogic}$

...

Every student knows some algorithm.

$\text{AliceIsStudent} \rightarrow \text{AliceKnowsBFS} \vee \text{AliceKnowsDFS} \vee \dots$

$\text{BobIsStudent} \rightarrow \text{BobKnowsBFS} \vee \text{BobKnowsDFS} \vee \dots$

...

- **First-order logic:** $\forall x \text{ Student}(x) \rightarrow (\exists y \text{ Algorithm}(y) \wedge \text{Knows}(x,y))$

Limitations of Propositional Logic (cont...)

- If you're adjacent to a pit, then you can feel breeze

- Propositional logic:**

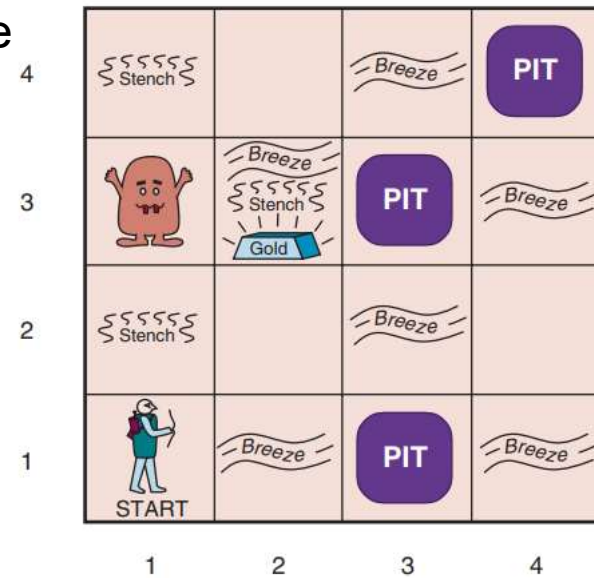
$$P_{1,1} \Rightarrow B_{1,2} \wedge B_{2,1}$$

$$P_{1,2} \Rightarrow B_{1,1} \wedge B_{2,2} \wedge B_{1,3}$$

...

- First-order logic:**

$$\forall w,x,y,z \quad \text{Adjacent}(\text{Grid}(w,x), \text{Grid}(y,z)) \wedge \text{Pit}(\text{Grid}(w,x)) \Rightarrow \text{Breeze}(\text{Grid}(y,z))$$





Limitations of Propositional Logic

- Propositional logic is sometimes clunky. What is missing?
- **Objects** and **predicates**: propositions (e.g., AliceKnowsLogic) have more internal structure (Alice, Knows, Logic)
- **Quantifiers** and **variables**: *all* is a quantifier which applies to each person, don't want to enumerate them all.



First-Order Logic

- ***Alice and Bob both know logic.***

$\text{Knows}(\text{Alice}, \text{Logic}) \wedge \text{Knows}(\text{Bob}, \text{Logic})$

- ***Every student knows logic.***

$\forall x, \text{Student}(x) \Rightarrow \text{Knows}(x, \text{Logic})$



Syntax and Semantics of FOL

- Terms (refer to object)

- Constant: Alice, Logic **$\text{Knows}(\text{Alice}, \text{Logic}) \wedge \text{Knows}(\text{Bob}, \text{Logic})$**
- Variable: x **$\forall x, \text{Student}(x) \Rightarrow \text{Knows}(x, \text{Logic})$**
- Function of terms: Father(.)

$\forall x, \text{HasBloodType}(x, \text{'AB'}) \Rightarrow \neg \text{HasBloodType}(\text{Father}(x), \text{'O'}) \wedge \neg \text{HasBloodType}(\text{Mother}(x), \text{'O'})$

- Predicate: Knows, Student, HasBloodType
- Atomic sentence: Predicate(Terms, ...)
- Complex sentence: $\neg$ sentence

sentence $\wedge$ sentence

sentence $\vee$ sentence

sentence $\Rightarrow$ sentence

sentence $\Leftrightarrow$ sentence

Quantifier variable, sentence



Syntax and Semantics of FOL (cont...)

- **Quantifiers**

- Universal quantifier $\forall$ (think conjunction): $\forall x P(x)$ is like $P(A) \wedge P(B) \wedge \dots$
- Existential quantifier $\exists$ (think disjunction): $\exists x P(x)$ is like $P(A) \vee P(B) \vee \dots$

- Properties of quantifiers

- $\neg \forall x P(x)$ is equivalent to $\exists x \neg P(x)$
- $\forall x \exists y \text{ Know}(x,y)$ and $\exists y \forall x \text{ Know}(x,y)$ are different



Translation from Natural Language to FOL

- **Every student knows logic.**

$$\forall x \text{ Student } x \Rightarrow \text{Knows}(x, \text{Logic})$$

- **Some student knows logic.**

$$\exists x \text{ Student } x \wedge \text{Knows}(x, \text{Logic})$$

- What about $\exists x \text{ Student } x \Rightarrow \text{Knows}(x, \text{Logic})$?



Translation from Natural Language to FOL

- There is some course that every student has taken.

$$\exists y \text{ Course}(y) \wedge [\forall x \text{ Student}(x) \rightarrow \text{Takes}(x, y)]$$

- Every even integer greater than 2 is the sum of two primes.

$$\forall x \text{ EvenInt}(x) \wedge \text{Greater}(x, 2) \rightarrow \exists y \exists z \text{ Equals}(x, \text{Sum}(y, z)) \wedge \text{Prime}(y) \wedge \text{Prime}(z)$$

- If a student takes a course and the course covers a concept, then the student knows that concept.

$$\forall x \forall y \forall z (\text{Student}(x) \wedge \text{Takes}(x, y) \wedge \text{Course}(y) \wedge \text{Covers}(y, z)) \rightarrow \text{Knows}(x, z)$$

Inference in FOL

- Convert everything to propositional logic
- Modus ponens
 - Sound
 - Complete for Horn clauses
- Resolution
 - Sound and complete

Recall: **Horn clause in PL**

$$\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_k \Rightarrow \beta$$

$$\alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_k \Rightarrow \text{False}$$

Each α_i is a propositional symbol

Horn clause in FOL

$$\forall x_1, \dots, \forall x_n \quad \alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_k \Rightarrow \beta$$

$$\forall x_1, \dots, \forall x_n \quad \alpha_1 \wedge \alpha_2 \wedge \cdots \wedge \alpha_k \Rightarrow \text{False}$$

Each α_i is an atomic sentence
(which may involve universal quantifier)

Forward Inference with Modus Ponens

$$\frac{\alpha'_1, \alpha'_2, \dots, \alpha'_k, \quad \forall x_1, \dots, \forall x_n, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta'}$$

where $\alpha'_1, \alpha'_2, \dots, \alpha'_k, \beta', \alpha_1, \dots, \alpha_k, \beta$ are atomic sentences, and $(\alpha_1, \alpha_2, \dots, \alpha_k, \beta)$ and $(\alpha'_1, \alpha'_2, \dots, \alpha'_k, \beta')$ can be **unified** through a **substitution** from variable to terms.

make them look the same

Forward Inference with Modus Ponens (cont...)

Take(Alice, SE350450)

Covers(SE350450, Logic)

$\forall x,y,z \text{ Take}(x,y) \wedge \text{Covers}(y,z) \Rightarrow \text{Knows}(x,z)$

Knowledge Base

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k, \quad \forall x_1, \dots, x_n, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta'}$$

$$\frac{\text{Take}(\text{Alice}, \text{SE350450}), \text{Covers}(\text{SE350450}, \text{Logic}), \forall x,y,z \text{ Take}(x,y) \wedge \text{Covers}(y,z) \Rightarrow \text{Knows}(x,z)}{?}$$

Substitution:

x / Alice

y / SE350450

z / Logic

? = **Knows(x,z)** applying the substitution
 = **Knows(Alice,Logic)**

Forward Inference with Modus Ponens (cont...)

$\forall w \text{ Take}(\text{Alice}, w)$

$\text{Covers}(\text{SE350450}, \text{Logic})$

$\forall x, y, z \text{ Take}(x, y) \wedge \text{Covers}(y, z) \Rightarrow \text{Knows}(x, z)$

Knowledge Base

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k, \quad \forall x_1, \dots, x_n, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta'}$$

$$\frac{\forall w \text{ Take}(\text{Alice}, w), \text{Covers}(\text{SE350450}, \text{Logic}), \forall x, y, z \text{ Take}(x, y) \wedge \text{Covers}(y, z) \Rightarrow \text{Knows}(x, z)}{?}$$

Substitution:

x / Alice

y / SE350450

z / Logic

w / SE350450

? = **Knows(x,z)** applying the substitution
 = **Knows(Alice, Logic)**

Forward Inference with Modus Ponens (cont...)

Take(Alice, SE350450)

$\forall v \text{ Covers}(\text{SE350450}, v)$

$\forall x, y, z \text{ Take}(x, y) \wedge \text{Covers}(y, z) \Rightarrow \text{Knows}(x, z)$

Knowledge Base

$$\frac{\alpha_1, \alpha_2, \dots, \alpha_k, \quad \forall x_1, \dots, x_n, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta}{\beta'}$$

$$\frac{\text{Take}(\text{Alice}, \text{SE350450}), \forall v \text{ Covers}(\text{SE350450}, v), \forall x, y, z \text{ Take}(x, y) \wedge \text{Covers}(y, z) \Rightarrow \text{Knows}(x, z)}{?}$$

Substitution:

x / Alice

y / SE350450

z / v

? = **Knows(x,z)** applying the substitution

= $\forall v$ **Knows(Alice,v)**

Substitute variable with another variable

Forward Inference with Modus Ponens (cont...)

Input: KB, α

If $\alpha \in \text{KB}$: **return** True

Repeat:

Choose a set of atomic sentences $\alpha'_1, \dots, \alpha'_k$ and rule $\forall x_1, \dots, \forall x_n, (\alpha_1 \wedge \alpha_2 \wedge \dots \wedge \alpha_k) \Rightarrow \beta$ in KB such that

$\alpha'_1, \alpha'_2, \dots, \alpha'_k, \beta'$ matches $\alpha_1, \alpha_2, \dots, \alpha_k, \beta$

under variable substitution, and β' cannot be subsumed by any sentence in KB.

If $\beta' = \alpha$: **return** True

If such matching does not exist: **return** False

Add β' to KB.

Forward Inference with Modus Ponens (cont...)

Take(Alice, SE350450)

Take(Alice, SE1234)

Take(Alice, HU4321)

Take(Bob, SE350450)

Covers(DSA, Search)

Covers(LinearAlgebra, matrix)

$\forall x,y,z \text{ Take}(x,y) \wedge \text{Covers}(y,z) \Rightarrow \text{Knows}(x,z)$

- How to find a matching between x,y,z and terms?

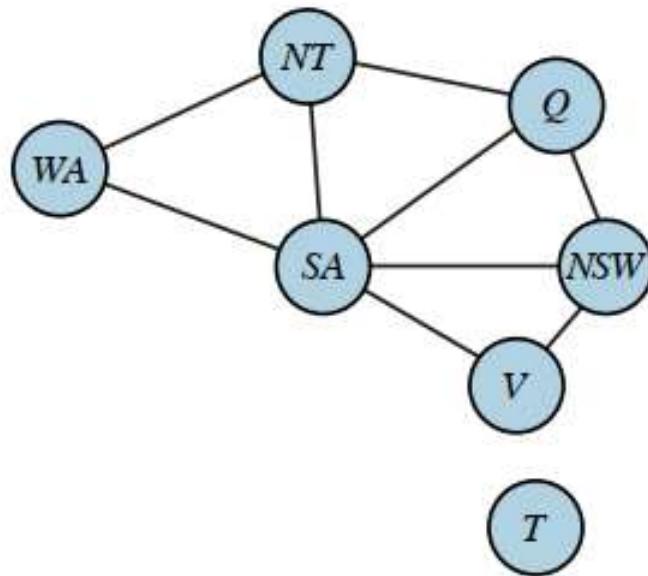
We can view of this problem as finding x,y,z that satisfies **constraints**

Take(x,y) and Covers(y,z)

→ Constraint Satisfaction Problem (**CSP**)

→ The heuristics we discussed before can be used

CSP is a Single Horn Clause



(a)

$$\begin{aligned} &Diff(wa, nt) \wedge Diff(wa, sa) \wedge \\ &Diff(nt, q) \wedge Diff(nt, sa) \wedge \\ &Diff(q, nsw) \wedge Diff(q, sa) \wedge \\ &Diff(nsw, v) \wedge Diff(nsw, sa) \wedge \\ &Diff(v, sa) \Rightarrow Colorable() \end{aligned}$$

$$\begin{aligned} &Diff(Red, Blue) \quad Diff(Red, Green) \\ &Diff(Green, Red) \quad Diff(Green, Blue) \\ &Diff(Blue, Red) \quad Diff(Blue, Green) \end{aligned}$$

(b)

Figure 9.5 (a) Constraint graph for coloring the map of Australia. (b) The map-coloring CSP expressed as a single definite clause. Each map region is represented as a variable whose value can be one of the constants *Red*, *Green*, or *Blue* (which are declared *Diff*).



Inference with Resolution

- High-level Ideas
 - Convert everything to CNF
 - Repeatedly apply the resolution rule from $KB \cup \{\neg\alpha\}$



Conversion to CNF

Anyone who likes all animals is liked by someone.

Input:

$$\forall x (\forall y \text{ Animal}(y) \rightarrow \text{Loves}(x, y)) \rightarrow \exists y \text{ Loves}(y, x)$$

Output:

$$(\text{Animal}(Y(x)) \vee \text{Loves}(Z(x), x)) \wedge (\neg \text{Loves}(x, Y(x)) \vee \text{Loves}(Z(x), x))$$

New to first-order logic:

- All variables (e.g., x) have universal quantifiers by default
- Introduce **Skolem functions** (e.g., $Y(x)$) to represent existential quantified variables



Conversion to CNF: Example (cont...)

Input:

$$\forall x (\forall y \text{Animal}(y) \rightarrow \text{Loves}(x, y)) \rightarrow \exists y \text{Loves}(y, x)$$

Eliminate implications (old):

$$\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee \exists y \text{Loves}(y, x)$$

Push $\neg$ inwards, eliminate double negation (old):

$$\forall x (\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee \exists y \text{Loves}(y, x)$$

Standardize variables (**new**):

$$\forall x (\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee \exists z \text{Loves}(z, x)$$



Conversion to CNF: Example (cont...)

$$\forall x (\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee \exists z \text{ Loves}(z, x)$$

Replace existentially quantified variables with Skolem functions (**new**):

$$\forall x [\text{Animal}(Y(x)) \wedge \neg \text{Loves}(x, Y(x))] \vee \text{Loves}(Z(x), x)$$

Distribute $\vee$ over $\wedge$ (old):

$$\forall x [\text{Animal}(Y(x)) \vee \text{Loves}(Z(x), x)] \wedge [\neg \text{Loves}(x, Y(x)) \vee \text{Loves}(Z(x), x)]$$

Remove universal quantifiers (**new**):

$$[\text{Animal}(Y(x)) \vee \text{Loves}(Z(x), x)] \wedge [\neg \text{Loves}(x, Y(x)) \vee \text{Loves}(Z(x), x)]$$

Resolution



Definition: resolution rule (first-order logic)

$$\frac{f_1 \vee \cdots \vee f_n \vee p, \quad \neg q \vee g_1 \vee \cdots \vee g_m}{\text{Subst}[\theta, f_1 \vee \cdots \vee f_n \vee g_1 \vee \cdots \vee g_m]}$$

where $\theta = \text{Unify}[p, q]$.



Example: resolution

$$\frac{\text{Animal}(Y(x)) \vee \text{Loves}(Z(x), x), \quad \neg \text{Loves}(u, v) \vee \text{Feeds}(u, v)}{\text{Animal}(Y(x)) \vee \text{Feeds}(Z(x), x)}$$

Substitution: $\theta = \{u/Z(x), v/x\}$.



Topics Covered

- *Artificial Intelligence: A Modern Approach (4th Edition) by Stuart Russell and Peter Norvig.* **(Chapter -07 (7.2-7.5),
Chapter-08(8.2-8.3),
Chapter-09(9.3-9.5))**

Any Question?